



Projet de fin de cours Deep Learning

Traduction multilingue de vidéos en français par Deep Learning

1. Contexte et objectif

Vous disposez d'un corpus de vidéos en français. Votre mission est de construire une chaîne de traitement (pipeline) permettant (i) d'extraire le texte (transcription), puis (ii) de traduire ce texte dans 4 à 5 langues, et (iii) d'évaluer et comparer plusieurs approches Deep Learning.

2. Compétences visées

- Comprendre et mettre en œuvre une pipeline audio/vidéo → texte → traduction.
- Choisir et justifier des modèles Deep Learning adaptés (séquentiels vs attentionnels).
- Mettre en place un protocole expérimental reproductible (splits, métriques, ablations).
- Réaliser une analyse quantitative et qualitative des résultats (erreurs, biais, limites).
- Produire des livrables professionnels : notebook, code commenté, slides, vidéo de démonstration.

3. Problématique

Problème central : "Comment traduire automatiquement des vidéos francophones en plusieurs langues en combinant transcription automatique (ASR) et traduction neuronale (NMT), et quelles architectures Deep Learning offrent le meilleur compromis qualité/robustesse/coût ?"

Les étudiants devront comparer au moins deux approches de traduction (idéalement trois), sur un même protocole.

4. Jeu de données (vidéos en français)

Multilingual TEDx (mTEDx) – Français

- Corpus de talks TEDx : audio/segments + transcriptions en français, avec traductions alignées vers plusieurs langues.
- Téléchargement (page OpenSLR, identifiant SLR100) : <http://www.openslr.org/100/>
- Exemple de ressources : mtedx_fr.tgz (français), et paires alignées mtedx_fr-en.tgz, mtedx_fr-es.tgz, mtedx_fr-pt.tgz.

5. Travail à réaliser (pipeline obligatoire)

5.1 Extraction du texte (ASR) depuis la vidéo

- Extraire l'audio des vidéos (ffmpeg).
- Transcrire en français avec un modèle ASR (au moins un).
- Produire des segments temporels et un fichier de sous-titres (SRT/VTT).
- Évaluer l'ASR si la vérité terrain existe (WER/CER) et analyser les erreurs (noms propres, chiffres, bruit).

5.2 Traduction multilingue (4–5 langues)

Vous traduirez la transcription française vers 4 ou 5 langues (ex : EN, ES, DE, AR, PT).

Vous comparerez 2 à 3 approches parmi celles ci-dessous.

6. Modèles à implémenter et comparer (2 à 3 approches)

Approche 1 – Baseline Seq2Seq LSTM (Encoder–Decoder)

- Construire un modèle NMT de base (tokenisation subword recommandée), entraîné/fine-tuné sur un sous-ensemble.
- Objectif : baseline séquentielle permettant de comprendre limites (dépendances longues, cohérence).

Approche 2 – Transformer NMT (MarianMT / modèle type T5/Marian)

- Utiliser un Transformer encoder–decoder pré-entraîné (ex : MarianMT), et le fine-tuner si possible.
- Objectif : comparer attention vs récursivité, et obtenir un niveau de qualité élevé.

Approche 3 – Modèle massivement multilingue (NLLB-200 / mBART / mT5)

- Utiliser un modèle multilingue de grande couverture et comparer aux modèles spécialisés.
- Objectif : mesurer la robustesse, notamment sur des langues moins proches du français.

7. Évaluation et analyse (obligatoire)

Vous devez fournir une évaluation quantitative ET une analyse qualitative approfondie.

7.1 Métriques

- ASR : WER et/ou CER (si transcript de référence disponible).
- Traduction : BLEU, chrF, COMET (si disponible), et/ou METEOR.
- Temps/ressources : temps d'inférence (par minute de vidéo), mémoire GPU/CPU.
- Analyse par conditions : segments longs vs courts, bruit, débit rapide, vocabulaire technique.

7.2 Analyse qualitative attendue

- Comparer des exemples de sorties sur les mêmes segments (au moins 10 segments).

- Typologie d’erreurs : contresens, omissions, hallucinations, termes propres, accords, ponctuation.
- Impact des erreurs ASR sur la traduction (cascade error).
- Discussion : pourquoi un modèle gagne/perd (architecture, tokenisation, domaine, taille).

8. Rendus (obligatoires)

1. Lien(s) vers le(s) jeu(x) de données utilisé(s) + description du sous-ensemble retenu (durée, nb segments, langues).
2. Notebook Jupyter (.ipynb) exécutable : chargement des données, prétraitement, entraînement/fine-tuning, évaluation, génération des sorties.
3. Code source complet, bien commenté, structuré (README, requirements/environment.yml, scripts).
4. Présentation PowerPoint (10–15 slides) : problème, données, modèles, protocole, résultats, analyse, limites.
5. Vidéo (5–8 minutes) : démonstration + explication des choix et des résultats (capture écran + voix).
6. Fichiers de sortie : au moins un exemple de sous-titres traduits (SRT/VTT) dans 4–5 langues.

9. Contraintes de reproductibilité

- Le notebook doit s’exécuter de bout en bout (au minimum sur un petit sous-ensemble).
- Fixer les seeds, documenter versions (Python, libs), et fournir un fichier requirements.txt ou environment.yml.
- Documenter le matériel utilisé (CPU/GPU, RAM) et les temps d’entraînement/inférence.
- Respecter la licence du dataset choisi et citer les sources dans le rapport/slides.

10. Plan de travail recommandé (12 jours)

- 4 jours : prise en main du dataset, extraction audio, baseline ASR + génération SRT.
- 4 jours : implémentation traduction (LSTM + Transformer), premières métriques, debugging.
- 4 jours : ajout modèle multilingue (option), évaluation complète, analyse, production livrables (PPT + vidéo).

Annexe – Liens utiles (rappel)

- mTEDx (OpenSLR SLR100) : <http://www.openslr.org/100/>
- VoxPopuli (GitHub) : <https://github.com/facebookresearch/voxpathuli>
- VoxPopuli (Hugging Face) : <https://huggingface.co/datasets/facebook/voxpathuli>
- Doc MarianMT (Transformers) : https://huggingface.co/docs/transformers/model_doc/marian
- NLLB (papier) : <https://arxiv.org/abs/2207.04672>